# Week 05 — Malware and Endpoint Security

> Understand malicious software and protect the devices people use. Slides: week05-malware-and-endpoint-security.pdf

**By the end you should be able to:**

1. Recognize major **malware categories** without focusing on malicious implementation.
2. Explain common **infection paths** and **endpoint protections**.
3. Choose **safe first actions** when a device may be compromised.

### Opening question

> A laptop suddenly becomes slow and starts opening unfamiliar windows. **What evidence would help distinguish malware from an ordinary technical problem?**

Useful evidence: **what processes are running**, **what the device is talking to on the network**, **what changed recently** (a new program, update or download), **scan results from endpoint protection**, and **startup/configuration changes**. A symptom on its own is not evidence — slowness also comes from a full disk, a failing battery or a bad update.

---

## 1. Recognize malware behavior

> Malware is identified by its **malicious purpose and actions**. Its **name, appearance, and visible symptoms can be misleading.** An innocent name does not guarantee safety.

**Malware** is software intended to cause harm. It may **steal information, disrupt services, spy on users, gain control, or enable later attacks**.

- **Some malware hides quietly.**
- **A symptom alone does not prove infection.**

## The categories, with the lecture's real examples

| Category | Real example in the slides | How it behaves | Controls named |
|---|---|---|---|
| **Virus** | **Sality** (file-infecting virus; Windows Defender detection) | Becomes active when an **infected file or program is executed**, and may **copy itself into other content** | Sharing infected files spreads it; modern threats combine several techniques |
| **Worm** | **Conficker** (network worm) | **Propagates across networks by exploiting weaknesses**, without waiting for every user to open a file | **Segmentation** limits spread; **fast patching** reduces exploitable paths |
| **Trojan** | **SocGholish** (fake browser update; the downloaded "Update.js" is a malicious script) | **Disguised** as a game, update, document, utility or pirated application | Download only from **trusted sources**; **appearance and filename do not prove safety** |
| **Ransomware** | **WannaCry** (ransom screen with countdown) | **Encrypts or blocks** systems, and may **also steal data** to increase pressure | **Payment does not guarantee recovery**; backups and response preparation reduce impact |
| **Spyware** | **Spyware.KeyLogger** (blocked by a security tool) | **Collects information** about user or device; a **keylogger captures keystrokes**, including sensitive ones | Often **no visible sign**; monitoring and endpoint protection help reveal it |
| **Bot** | **Neutrino Bot** (ordinary-looking file found in AppData) | Lets an **external operator use the device** for spam, fraud, attacks or further access | Most users never see a "bot" screen; hidden activity consumes data and resources |

**The distinction that matters most: virus vs worm.** A **virus needs a user action** (running or sharing an infected file). A **worm spreads by itself** through reachable, unpatched systems. WannaCry is remembered because it was **ransomware that also spread as a worm**.

## Quick check

> **Can a computer be infected even when no warning window appears? Explain.**

**Yes.** Spyware, bots and many Trojans are designed to stay **hidden** — showing a window would defeat their purpose. Only ransomware wants to be seen. Detection comes from **scans, process and network monitoring, and configuration changes**, not from waiting for a popup.

## Activity: name the likely malware type

> For each observation, guess the most likely type and say **what more information you would need to be sure**.

| # | What you observe | Most likely type | What else you'd want |
|---|---|---|---|
| 1 | An infected program is shared on a USB drive; other executable files become infected | **Virus** | Confirmation that files were modified, and that infection followed execution |
| 2 | Several unpatched computers become infected **without each user opening a file** | **Worm** | Which service was exploited, and whether patches were missing |
| 3 | A "free game" installs an unwanted **remote-control** program | **Trojan** (leading to remote access) | The installer's source and publisher, and what it installed |
| 4 | Course files no longer open; a screen demands payment | **Ransomware** | Whether files are encrypted, and whether backups are intact |
| 5 | A program secretly records what a student types | **Spyware / keylogger** | Scan results, and what data left the device |
| 6 | A device sends attack traffic when a **remote operator** tells it to | **Bot** | Network logs showing command-and-control traffic |
| 7 | A laptop is slow, but **no scan, process or network evidence** is available | **Unknown — possibly not malware at all** | Any actual evidence: this is the opening question again |
| 8 | A fake update steals information **and downloads ransomware** | **Trojan + spyware + ransomware** | The infection path, and what was taken before encryption |

*(The slides pose the cases without printing answers; these follow from the category table.)* Case 7 is the real lesson: **without evidence, "slow laptop" is not a diagnosis.** Case 8 shows that **one incident can involve several categories**.

---

## 2. Break the infection path

> Malware often depends on **unsafe content, weak configuration, excessive privilege, or unpatched software**.

| Path | How it works | Controls |
| --- | --- | --- |
| **Malicious messages** | Attachments, links, shared documents and fake login pages lead to harmful downloads or credential theft | **Verify unexpected content**; automatic filtering and user reporting |
| **Untrusted downloads** | Pirated software, unofficial app stores, fake updates, bundled installers | Use **approved sources and software lists**; check **publisher and update path** |
| **Unpatched vulnerabilities** | Attackers exploit **known flaws** when updates are delayed or software is unsupported | Prioritize **internet-facing and critical** systems; **replace software that no longer receives fixes** |
| **Excess privilege** | Malware with **administrator access** can change more settings, reach more data, disable more controls | Use **standard accounts** for daily work; approve elevated actions carefully |
| **Removable media** | Unknown or infected **USB devices** introduce malicious files or unauthorized hardware | Follow organizational **media policy**; **do not connect found or unknown devices** |
| **Weak configuration** | **Automatic execution, exposed services, default credentials, disabled protections** make infection and persistence easier | Use **secure baselines**; monitor for unexpected configuration changes |
| **Supply chain** | Legitimate **updates, packages or vendors** become a path when their development or delivery is compromised | Use **trusted signed sources**; track **dependencies and supplier notices** |

## Quick check

> **Why can an attachment from a known contact still be dangerous?**

Because the **contact's account may be compromised**, their address may be **spoofed**, or they may be forwarding something harmful **without knowing**. This is Week 3's point in a new form: **recognition is not authorization**, and a familiar sender is not a check.

## Activity: trace four infection stories

> Trace each story from **initial contact** → **execution** → **privilege** → **persistence** → **impact**, and place possible controls.

| Story | Initial contact | Execution | Privilege | Persistence | Impact | Possible control |
|---|---|---|---|---|---|---|
| **1. Malicious email attachment** | User receives an infected document | User **opens the attachment** | Macro/script runs as the user; may seek admin | Startup entry or scheduled task | Data theft, further downloads | Message filtering, verify unexpected content, standard accounts |
| **2. Fake software / update** | User downloads a fake game or update | User **runs the installer** | Installer asks for elevation | Installed program, auto-start | Remote control, information theft | Approved sources, check publisher, restrict installation |
| **3. Network worm** | Vulnerable computer is **reachable on the network** | Worm **exploits an unpatched service** — no user action | Runs with the service's privileges | Service or system component | Rapid spread, disruption | **Patch quickly**, **segmentation**, limit exposed services |
| **4. Infected USB** | User connects a USB with an infected program | User **runs the infected file** | Runs as the user | Copies itself to other files/media | Spread across machines, data loss | Media policy, disable auto-run, don't connect unknown devices |

The columns are the exam structure: **contact** → **execution** → **privilege** → **persistence** → **impact**. Story 3 is the one with **no user action**, which is what makes a worm a worm.

---

## 3. Protect and respond at the endpoint

> Secure configuration, updates, least privilege, detection, backups, and fast reporting **work together**.

| Control | Detail |
|---|---|
| **Updates** | OS, browsers, applications, **firmware** and security tools should get supported updates through trusted mechanisms. Enable automatic updates where appropriate; **track devices that cannot update** |
| **Endpoint protection** | Antivirus and endpoint tools block known threats and identify suspicious files, processes, settings or communications. **Detection quality changes over time**, and **alerts need investigation** |
| **Secure configuration** | Disable unnecessary services, restrict installation, lock screens, encrypt storage, use approved software. **Defaults prioritize convenience**, and configuration should be **repeatable and reviewable** — every computer follows the same rules |
| **Backups** | **Protected, separated and tested** backups restore information **without relying on the attacker**. **A connected backup may also be encrypted. Test recovery before an incident** |

### Suspicious devices should be reported quickly

If you suspect infection: **stop sensitive work**, **avoid unnecessary changes**, and **report quickly** so trained staff can decide what to do.

- **Do not continue experimenting on the device** — don't keep clicking around, opening files, running random scans or trying to "see what happens".
- **Follow guidance before disconnecting critical systems.** For servers and lab systems, don't just shut them down: IT/security staff may need to **preserve evidence**.

### Recovery must return to a trusted state

> Recovery is complete only when the system is **clean**, the **weakness is fixed**, **credentials are safe**, **data is restored**, and the system is **monitored afterwards**.

- **Restoring files alone may restore the weakness.** Restore clean data **and** fix the cause.
- **Document the cause and the preventive improvement.**

### Quick check

> **Why is antivirus useful but insufficient as the only endpoint control?**

**No tool detects every threat.** Protection also depends on **configuration, updates, access/privilege, backups, and users**. This is **defense in depth** from Week 1: any single layer can fail.

### Six questions before recommending a design

1. Observed **symptom**
2. Possible **infection path**
3. **Privilege** level
4. **Affected data**
5. **Safe first action**
6. **Recovery evidence**

### Activity: a suspicious library computer

> A fake update was installed. The browser opens unknown pages. A student used **email** after the installation.

- **Possible malware behavior:** a **Trojan** from the fake update (SocGholish-style), now showing **adware/redirect** behavior, with possible **spyware** collecting what was typed.
- **Immediate reporting and containment:** stop using the machine for anything sensitive, **don't experiment on it**, report to university IT through the official route, and follow their guidance on isolating it so evidence is preserved.
- **Affected accounts:** any account used **after the installation** — certainly the **email account** (and anything signed into from that browser). Passwords and sessions should be reset **from a different, trusted device**.
- **Controls to prevent recurrence:** restrict installation on shared machines, **standard (non-admin) accounts**, approved software sources, endpoint protection with monitored alerts, secure baselines for lab computers, and user guidance on fake updates.

---

## Q & A: check your understanding

**1. What is malware?** Software intentionally designed to cause harm, steal information, disrupt service, or enable unauthorized control.

**2. How does a virus commonly spread?** Through **execution and sharing** of infected files or programs.

**3. How is a worm different from a virus?** A worm can **propagate through networks without depending on every user** to open an infected file.

**4. What is a Trojan?** Malware **disguised** as legitimate or useful software or content.

**5. What is ransomware?** Malware that **blocks or encrypts** systems or data to **demand payment**, often combined with data theft.

**6. Why do updates matter?** They **correct known vulnerabilities** that malware and attackers may exploit.

**7. Why does least privilege help?** It **limits what malicious code can change or access**.

**8. Why is antivirus not enough?** No tool detects every threat, and protection also depends on configuration, updates, access, backups, and users.

**9. What should a user do after noticing suspicious behavior? Stop sensitive activity** and **report** through the approved support process.

**10. What makes a backup useful after malware?** It is **protected from the incident**, **recent enough**, and **tested** for successful restoration.

---

## Assignment: trace a malware infection

> 1. Choose **one** scenario: malicious email attachment, fake software/update, network worm, or infected USB.
> 2. Create one attack story using this path: **Initial contact** → **Execution** → **Privilege** → **Persistence** → **Impact** → **Recovery**

My submission: assignment/submission.pdf (markdown source) — the *fake software / update* scenario.

Guidance: keep the six stages as headings so the structure is obvious, and put a **control** against each stage rather than all at the end. Finish with the **trusted state** checklist (clean system, weakness fixed, credentials safe, data restored, monitoring) instead of stopping at "restore the files".

**Sources from the slides:** Microsoft (Defender detection; WannaCrypt 12 May 2017), Malwarebytes (Conficker; SocGholish; Spyware.KeyLogger; Neutrino Bot), CISA (StopRansomware).

---

## Most important for exams

- **Five ideas the lecture ends on:** categories describe **different spreading methods and harmful goals**; **messages, downloads, vulnerabilities, media, privileges and suppliers** are infection paths; **updates, secure configuration, least privilege and endpoint protection** reduce opportunity; **protected and tested backups** reduce ransomware impact; **fast reporting and trusted recovery matter more than guessing the exact malware name**.

- **Six categories with one-line definitions and an example each:** virus (Sality), worm (Conficker), Trojan (SocGholish), ransomware (WannaCry), spyware/keylogger, bot (Neutrino).
- **Virus needs user action; worm spreads by itself.** WannaCry = ransomware **that spread as a worm**.
- **Malware is defined by purpose and actions, not by name or appearance.** Infection can be **completely invisible**; a symptom is not proof.
- **Seven infection paths** and one control for each.
- **Infection story structure:** initial contact → execution → privilege → persistence → impact (→ recovery).
- **Least privilege** limits damage; **excess privilege** multiplies it.
- **Backups:** protected, **separated** (a connected backup can be encrypted too), recent, and **tested**.
- **Antivirus is not enough** — defense in depth.
- **First actions on a suspected device:** stop sensitive work, **don't experiment**, report fast, follow guidance before disconnecting critical systems (evidence).
- **Recovery = trusted state:** clean system, weakness fixed, credentials safe, data restored, monitoring afterwards. **Restoring files alone can restore the weakness.**